

MACHINE LEARNING COURSE · CS-433

Loss Functions

September 9, 2026

Martin Jaggi

credits to Mohammad Emtiyaz Khan



Motivation

Consider the following models.

1-parameter model: $y_n \approx w_0$

2-parameter model: $y_n \approx w_0 + w_1 x_{n1}$

How can we **estimate** (or guess) values of w given the data D ?

What is a loss function?

A **loss function** (or energy, cost, training objective) is used to learn parameters that explain the data well.

The loss function quantifies how well our model does — or, in other words, how costly our mistakes are.

Two desirable properties of loss functions

When the target y is real-valued, it is often desirable that the cost is symmetric around 0, since both positive and negative errors should be penalized equally.

Also, our cost function should penalize “large” mistakes and “very-large” mistakes similarly.

Statistical vs. computational trade-off

If we want better statistical properties, then we have to give up good computational properties.

Statistical

Does the loss cope with outliers, so the fit is not dragged around by a few odd measurements?

Computational

Is the loss convex, so that we can actually find its minimum?

We meet both sides of this trade-off in the rest of the lecture.

Mean Square Error (MSE)

MSE is one of the most popular loss functions.

$$\text{MSE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N [y_n - f_{\mathbf{w}}(\mathbf{x}_n)]^2$$

Does this loss function have both mentioned properties?

An exercise for MSE

Compute MSE for the 1-param model: $\mathcal{L}(w_0) := \frac{1}{N} \sum_{n=1}^N [y_n - w_0]^2$

	1	2	3	4	5	6	7
$y_1 = 1$							
$y_2 = 2$							
$y_3 = 3$							
$y_4 = 4$							
$\text{MSE}(w) \cdot N$							
$y_5 = 20$							
$\text{MSE}(w) \cdot N$							

Some help: $19^2 = 361$, $18^2 = 324$, $17^2 = 289$, $16^2 = 256$, $15^2 = 225$, $14^2 = 196$, $13^2 = 169$.

Outliers

Outliers are data examples that are far away from most of the other examples. Unfortunately, they occur more often in reality than you would want them to!

MSE is not a good loss function when outliers are present.

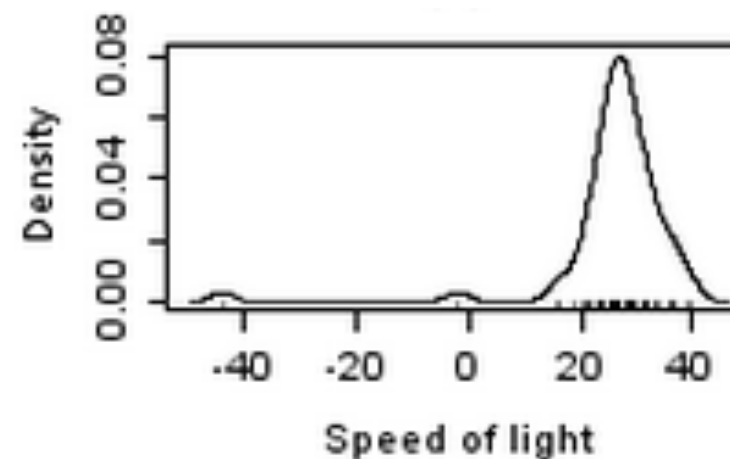
Handling outliers well is a desired *statistical* property.

Outliers: a real example

Speed of light measurements — from Gelman's book on Bayesian data analysis.

28	26	33	24	34	-44	27	16	40	-2
29	22	24	21	25	30	23	29	31	19
24	20	36	32	36	28	25	21	28	29
37	25	28	26	30	32	36	26	30	22
36	23	27	27	28	27	31	27	26	33
26	32	32	24	39	28	24	25	32	25
29	27	28	29	16	23				

(a) Original speed of light data collected by Simon Newcomb.



(b) Histogram showing outliers.

Note the measurements far from the bulk of the data (-44 and -2), and the matching bumps in the histogram. Handling outliers well is a desired *statistical* property.

Mean Absolute Error (MAE)

$$\text{MAE}(\mathbf{w}) := \frac{1}{N} \sum_{n=1}^N |y_n - f_{\mathbf{w}}(\mathbf{x}_n)|$$

Repeat the exercise with MAE.

	1	2	3	4	5	6	7
$y_1 = 1$							
$y_2 = 2$							
$y_3 = 3$							
$y_4 = 4$							
$\text{MAE}(w) \cdot N$							
$y_5 = 20$							
$\text{MAE}(w) \cdot N$							

Convexity

Roughly, a function is **convex** iff a line segment between two points on the function's graph always lies above the function.

A function $h(u)$ with $u \in \mathbb{R}^D$ is **convex**, if for any $u, v \in \mathbb{R}^D$ and for any $0 \leq \lambda \leq 1$, we have:

$$h(\lambda \mathbf{u} + (1 - \lambda) \mathbf{v}) \leq \lambda h(\mathbf{u}) + (1 - \lambda) h(\mathbf{v})$$

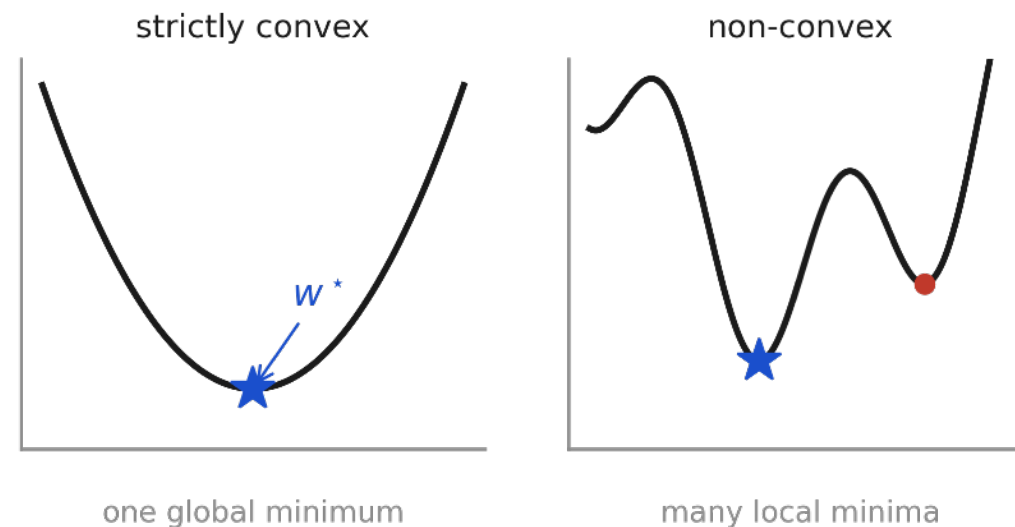
A function is **strictly convex** if the inequality is strict.

Importance of convexity

A strictly convex function has a unique global minimum w^* . For convex functions, every local minimum is a global minimum.

Sums of convex functions are also convex.
Therefore, MSE combined with a linear model is convex in w .

Convexity is a desired *computational* property.



Exercise

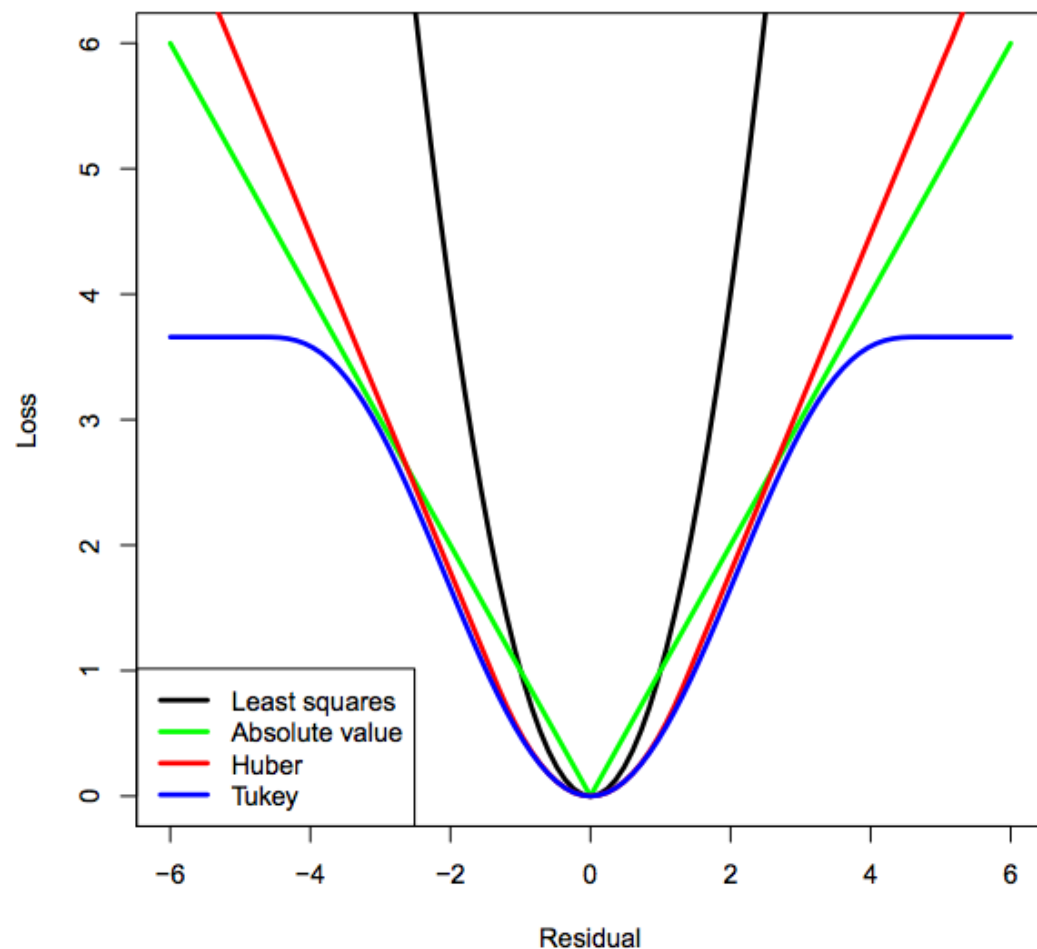
Can you prove that fitting a linear model with MAE loss is convex?

As a function of the parameters $w \in \mathbb{R}^D$, for linear regression:

$$f_{\mathbf{w}}(\mathbf{x}) := f(\mathbf{x}, \mathbf{w}) := \mathbf{x}^\top \mathbf{w}$$

Computational vs. statistical trade-off

So which loss function is the best?



If we want better statistical properties, then we have to give up good computational properties.

Figure taken from Patrick Breheny's slides.

Other loss functions

Huber loss

$$\text{Huber}(e) := \begin{cases} \frac{1}{2}e^2 & \text{if } |e| \leq \delta \\ \delta|e| - \frac{1}{2}\delta^2 & \text{if } |e| > \delta \end{cases}$$

Huber loss is convex, differentiable, and also robust to outliers. However, setting δ is not an easy task.

Tukey's bisquare loss (defined in terms of the gradient)

$$\frac{\partial \mathcal{L}}{\partial e} := \begin{cases} e \{1 - e^2/\delta^2\}^2 & \text{if } |e| \leq \delta \\ 0 & \text{if } |e| > \delta \end{cases}$$

Tukey's loss is non-convex, but robust to outliers.

Further reading

On outliers

- Wikipedia page on “Robust statistics”.
- Repeat the exercise with MAE.
- Sec. 2.4 of Kevin Murphy’s book, for an example of robust modeling.

Nasty loss functions: visualization

See Andrej Karpathy’s Tumblr post for many loss functions gone “wrong” for neural networks: [lossfunctions.tumblr.com](https://karpathy.github.io/2015/07/25/lossfunctions/)